HGraphScale: Hierarchical Graph Learning for Autoscaling Microservice Applications in Container-Based Cloud Computing

Zhengxin Fang, *Graduate Student Member, IEEE*, Hui Ma, *Senior Member, IEEE*, Gang Chen, *Senior Member, IEEE*, and Rajkumar Buyya, *Fellow, IEEE*

***Abstract*—Microservice architecture has become a dominant paradigm in application development due to its advantages of being lightweight, flexible, and resilient. Deploying microservice applications in the container-based cloud enables fine-grained elastic resource allocation. Autoscaling is an effective approach to dynamically adjust the resource provisioned to containers. However, the intricate microservice dependencies and the deployment scheme of the container-based cloud bring extra challenges of resource scaling. This article proposes a novel autoscaling approach named HGraphScale. In particular, HGraphScale captures microservice dependencies and the deployment scheme by a newly designed hierarchical graph neural network, and makes effective scaling actions for rapidly changing user requests workloads. Extensive experiments based on real-world traces of user requests are conducted to evaluate the effectiveness of HGraphScale. The experiment results show that the HGraphScale outperforms existing state-of-the-art autoscaling approaches by reducing at most 80.16% of the average response time under a certain VM rental budget of application providers.**

***Index Terms*—Autoscaling, microservice application, container-based cloud, graph neural network, deep reinforcement learning.**

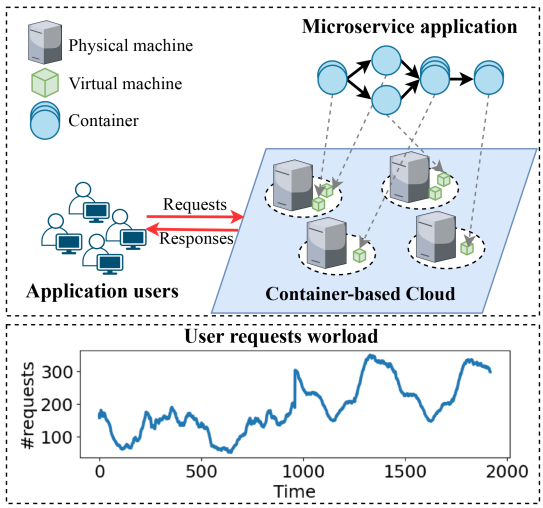

Fig. 1. Microservice application deployed in the container-based cloud with fluctuating user requests.

I. Introduction

MICROSERVICE applications are marking a paradigm shift in how software systems are designed and managed [8]. These modern applications are composed of lightweight and scalable microservices, which improve scalability, agility, and resilience [7]. Each microservice is instantiated by one or more containers. Building on this paradigm, cloud computing serves as a critical enabler for hosting and scaling microservice applications [1].

The dynamic resource adjustment in cloud computing, known as *autoscaling* [4], [28], [30], [53], empowers microservice applications to efficiently handle fluctuating user requests [51] by leveraging *horizontal scaling* and *vertical scaling* techniques. Horizontal scaling creates or deletes replicas of containers, while vertical scaling adjusts the resources (e.g., CPU) provisioned to individual containers.

The effectiveness of autoscaling is further enhanced by the container-based cloud [14], [21], [46], which offers fine-grained resource allocation tailored to dynamic workloads. As illustrated in Fig. 1, microservice applications deployed in the container-based cloud follow a hierarchical structure: containers are hosted within Virtual Machine instances (VMs), which, in turn, are deployed on Physical Machine instances (PMs).

Within the container-based cloud, the Quality of Service (QoS) of microservice applications, such as their average response time [11], depends on the number of containers and the resources provisioned to them [4], [44], [57]. To maintain high QoS under fluctuating user requests workload (as shown in Fig. 1), this article investigates the critical problem of *Autoscaling Microservice applications in the Container-based cloud*, referred to as the AMC problem in the remaining of this paper.

In the AMC problem, provisioning excessive resources to containers may help meet the service level objectives (SLOs)

Received 3 October 2025; revised 30 December 2025; accepted 31 December 2025. Date of publication 12 January 2026; date of current version 5 February 2026. This work was supported by the China Scholarship Council (CSC)/Victoria University Scholarship. *(Corresponding author: Zhengxin Fang.)*

Zhengxin Fang, Hui Ma, and Gang Chen are with the School of Engineering and Computer Science & Centre for Data Science and Artificial Intelligence, Victoria University of Wellington, Wellington 6140, New Zealand (e-mail: zhengxin.fang@ecs.vuw.ac.nz; hui.ma@ecs.vuw.ac.nz; aaron.chen@ecs.vuw.ac.nz).

Rajkumar Buyya is with the School of Computing and Information Systems, University of Melbourne, Melbourne, VIC 3010, Australia (e-mail: rbuyya@unimelb.edu.au).

Digital Object Identifier 10.1109/TSC.2026.3651267

1939-1374 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:11 UTC from IEEE Xplore. Restrictions apply.

of application providers but often leads to significant resource wastage [3], [37]. The cloud resources wastage increases cost for application providers due to unnecessary VM rentals [25], [44], [57]. Additionally, resource constraints at the PM and VM levels further increase the complexity of container resource provisioning and autoscaling.

Given the above challenges, an effective autoscaling approach is essential to enhance the QoS of applications while adhering to a defined cost budget. However, many existing approaches rely on simple threshold-based mechanisms, such as Amazon auto-scaling service [2] and Horizontal Pod Autoscaler (HPA) [9]. These methods make scaling actions based on a pre-defined threshold. Nonetheless, manually selecting a threshold for changing workload is non-trivial. An inappropriate threshold can easily result in under-provisioning, leading to QoS degradation, or over-provisioning, causing unnecessarily high costs [26].

To address the limitations of threshold-based approaches, Deep Reinforcement Learning (DRL) is a promising approach for autoscaling [4], [31], [33], [34]. It can automatically learn generalizable scaling policies that adapt to dynamic environments. DRL-based methods employ deep neural networks, such as Graph Neural Networks (GNNs), to obtain container embeddings. These embeddings capture implicit characteristics and complex dependencies among containers, which are then used to guide scaling decisions. However, it is not intuitive to design an effective embedding learning approach for the AMC problem since the container-based cloud is a rather complex system. Two major issues have not been addressed by existing studies.

First, existing DRL-based autoscaling approaches [4], [34], [44] do not explicitly consider the *deployment scheme* of the container-based cloud [10]. The deployment scheme models the hierarchical placement of containers to VMs and VMs to PMs, together with network communication among containers. Ignoring such deployment information leads to suboptimal autoscaling decisions, increased physical network communication overhead, inefficient resource utilization, and ultimately degraded QoS. However, existing approaches only focus on the features of individual components, such as containers, VMs and PMs. The deployment scheme is critical because different deployment schemes result in varying resource constraints (e.g., the capacities of VMs and PMs), which directly influence the effectiveness of scaling actions. Ignoring this factor can lead to suboptimal scaling actions and resource utilization.

Second, GNNs employed in existing autoscaling approaches [33], [47] aggregate information in a *flat* way. In this flat structure, containers, VMs and PMs are modeled in a single layer. As a result, long-range dependencies (e.g., between containers on different PMs) require many message-passing steps to capture. Studies [32], [55], [61] have shown that the flat GNN structure cannot effectively capture long-range dependencies for learning node embeddings. This issue becomes more aggravated in the AMC problem with the increasing number of containers, VMs and PMs in the container-based cloud.

To address the above issues, this paper focuses on designing a novel embedding learning approach to improve the performance of the DRL-based autoscaling approach. For this purpose, we construct a three-layer hierarchical graph. It models both the dependencies among containers and the deployment scheme. The hierarchical structure of this graph, from bottom to top, is: *PM layer*, *VM layer* and *container layer*. Then, we design a novel Hierarchical Graph Neural Network (HGNN) to solve the issue of long-range information aggregation.

HGNN is a solution for the issue of long-range information aggregation [32], [61]. However, existing HGNN approaches [32], [59] mainly learn whole-graph embeddings by aggregating information in a fine-to-coarse manner. These approaches are effective for tasks that require holistic graph representations. However, it is not well-suited for the AMC problem, where precise scaling actions depend on embeddings at the granularity of individual containers rather than the entire graph.

To fill this gap, we propose a *Cloud-oriented Hierarchical Graph Neural Network (CHGNN)*, which is an HGNN designed to effectively learn container embeddings from a cloud environment. Unlike traditional methods, CHGNN first aggregates information locally within lower-layer nodes and then propagates it to higher layers. This *bottom-up information aggregation mechanism* establishes shortcut connections [22], [32], [55] between distant nodes in the graph, enabling effective processing of global context. Consequently, CHGNN can capture comprehensive global information from the container-based cloud for the container layer. This mechanism not only represents a departure from existing HGNN paradigms but also delivers a more precise and scalable solution for the AMC problem.

Through developing CHGNN, this paper makes the following main contributions:

- We represent the container-based cloud as a three-layer hierarchical graph. Meanwhile, we design a novel HGNN, i.e., CHGNN, to learn container embedding from the hierarchical graph. To our knowledge, this is the first work to learn embedding for autoscaling using HGNN, allowing to make more effective scaling actions for the AMC problem than existing approaches.
- We propose a novel bottom-up information aggregation mechanism for CHGNN to effectively capture thorough global information from the container-based cloud. This mechanism provides an accurate and scalable autoscaling solution to the AMC problem.
- We propose a novel DRL-based autoscaling approach that leverages CHGNN with a bottom-up information aggregation mechanism to effectively learn container embeddings. In addition, a newly designed *scaling policy network* is employed to make scaling decisions. We name this autoscaling approach as HGraphScale. Experiment results based on real-world traces indicate that HGraphScale can outperform five state-of-the-art autoscaling approaches.

The rest of this article is organized as follows. Section II presents the literature review of existing autoscaling approaches. Section III presents formal problem definitions of the problem. Section IV gives details of HGraphScale for autoscaling. The experiment designs, results and further analysis are shown in Section V. At last, Section VI makes conclusions and gives potential future directions.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:11 UTC from IEEE Xplore. Restrictions apply.

II. Related Work

In this section, we review existing autoscaling approaches for microservice applications.

A. Heuristic-Based Autoscaling

AWS-Scale [2] and Horizontal Pod Auto-scaler (HPA) [9] are autoscaling techniques that rely on manually determined thresholds. For example, the resources provisioned to containers are increased if the resource utilization is higher than a given threshold; otherwise, it decreases the resources provisioned to containers. However, manually designing a threshold for changing workload is challenging.

To address the above issue, some heuristic-based autoscaling approaches are proposed to make scaling actions based on predicted future workload. ProScale [11] is a proactive autoscaling method that leverages the accurate and fast Simple Moving Average (SMA) to predict future request workloads. Then, the resource adjustment of containers is based on a greedy method. PBScaler [57] is proposed to detect the bottleneck microservices in an application. Subsequently, a genetic algorithm is applied to decide the number of containers required by bottleneck microservices. StatusScale [53] is a status-aware autoscaling approach that selects appropriate autoscaling strategies for resource scheduling based on load status.

The above autoscaling methods require substantial human efforts to design the heuristics or fine-tune the thresholds. Meanwhile, the heuristics methods exhibit poor generalization ability in dynamically changing environments [58].

B. Reinforcement Learning-Based Autoscaling

Existing studies [4], [19], [20], [34], [44], [60] have shown that the RL-based autoscaling methods can effectively adjust the resource allocation to handle the changing workload. For example, A-SARSA is proposed [60] to combine neural network based workload prediction and the SARSA algorithm [52] to make scaling actions based on predicted workload.

A Q-learning based autoscaling approach [19] is proposed for workflow autoscaling, which considers the workflow structure when making scaling actions. [20] further compared the performance of Q-learning and SARSA for workflow autoscaling, considering the workflow structures. Their results show that SARSA can achieve significantly better performance in many scenarios compared to Q-learning.

The above approaches use table-based RL techniques, struggling to handle high-dimensional state spaces. To tackle this limitation, the DRL-based autoscaling approach has been gaining more attention in recent years. For instance, a Deep Q-Network (DQN) [35] based autoscaling method, called HRA [34], is proposed to make holistic autoscaling actions for microservice applications. Similarly, DeepScale [44] integrates DQN and heuristics methods to make scaling actions for applications. DRPC [4] is a TD3 [18] based DRL approach to make scaling actions based on an embedding learned by multiple distributed neural networks. ASTRA [30], a recently introduced approach, leverages an adversarial DRL algorithm for autoscaling.

The above DRL-based approaches fail to explicitly consider the deployment scheme, which impacts scaling actions. This hinders the effectiveness of these methods in addressing the AMC problem.

C. Graph Neural Network-Based Autoscaling

Beyond heuristic and DRL-based autoscaling methods, GNN-based approaches have emerged as a popular solution for autoscaling. For instance, DeepScaler [33] is proposed to estimate resource utilization by GNN, which is further utilized to guide the autoscaling decisions. GRAF [37], [38] is proposed to predict the tail latency of microservice applications. The predicted latency is leveraged for proactive autoscaling decision-making. AGQ [31] is an autoscaling approach that utilizes a GNN-based resource usage predictor, which directly informs the autoscaler's scaling decisions.

In summary, existing GNN-based autoscaling approaches are designed for prediction tasks, either forecasting resource usage or predicting latency. Such prediction tasks require large datasets for training, and these GNNs do not account for the deployment scheme in the cloud environment.

D. Summary

To address the above limitations of existing autoscaling approaches, this article proposes HGraphScale, a novel DRL-based autoscaling approach that incorporates a newly designed GNN and information aggregation mechanism. The details of comparison between HGraphScale and other DRL-based and GNN-based autoscaling approaches are shown in Table I

III. Problem Description

In this section, we formulate the problem of Autoscaling Microservice application in the Container-based cloud (the AMC problem).

A. System Overview

Fig. 2 presents the system model of the container-based cloud, where containers of each microservice application are deployed on VMs, which are further hosted by PMs. A microservice application can be modeled as a Directed Acyclic Graph (DAG) $App = \langle V_{app}, E_{app} \rangle$, as shown in Fig. 2. $V_{app} = \{ms_0, ms_1, \ldots, ms_n\}$ represents n microservices. $e_{ij}^{app} \in E_{app}$ denotes the execution dependency between a pair of adjacent microservices ms_i and ms_j. Following existing studies [5], [14], [50], [62], a microservice ms_i is instantiated by at least one container Con_i^j, where j denotes the index of the container.

Each user request triggers the execution of a workflow instance $WF = \langle V_{wf}, E_{wf} \rangle$ [50]. $V_{wf} = \{t_{start}, t_1, t_2, \ldots, t_n, t_{end}\}$ denotes tasks in a user request, where t_{start} and t_{end} are dummy starting and ending tasks, respectively. $e_{ij}^{wf} \in E_{wf}$ represents t_i is the predecessor task of t_j while t_j is the successor task of t_i. Task t_i can only be executed by a container of the corresponding microservice ms_i.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:11 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF HGRAPHSCALE WITH DRL-BASED AND GNN-BASED AUTOSCALING APPROACHES

	Approaches										
	[31]	[19]	[34]	[44]	[4]	[20]	[37]	[33]	[38]	[31]	**Ours**
Vertical scaling				✓	✓						✓
Horizontal scaling	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
QoS improvement	✓	✓	✓		✓	✓	✓	✓	✓	✓	✓
Cost saving	✓		✓	✓		✓		✓		✓	✓
High dimensional state			✓	✓	✓		✓	✓	✓	✓	✓
Microservice dependency	✓					✓	✓	✓	✓	✓	✓
Deployment scheme											✓

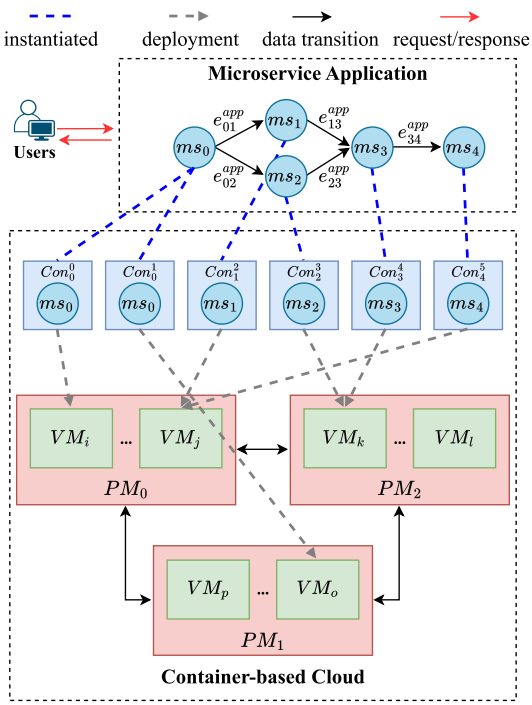

Fig. 2. An example microservice application.

B. QoS and Cost Modeling

Let et_i denote the execution time of t_i with one vCPU, and $concpu_i^j$ denotes the amount of vCPU provisioned to Con_i^j. This study focuses on resource adjustment of vCPU [12], [38], [44]. This is because existing studies showed that CPU is the dominant factor affecting microservice application response time [29], [38], [53]. Accordingly, the execution time of t_i in Con_i^j is

$$ET_i^j = \frac{et_i}{concpu_i^j}. \tag{1}$$

As assumed in existing studies [5], [42], [50], [58], a container can execute at most one task at any time. Meanwhile, each container maintains a pending queue of task, following [42], [44]. Each task starts execution only after the preceding task in the queue has been completed. As a result, the finish time FT of a task t_i in Con_i^j is calculated by:

$$FT_i^j = ST_i^j + ET_i^j, \tag{2}$$

where ST_i^j indicates the start time of t_i in Con_i^j. Particularly, Particularly, ST_i^j is defined as

$$ST_i^j = WT_i^j + FT^{pre_i} + TT^{pre_i}, \tag{3}$$

where WT_i^j is the waiting time of t_i in the pending queue of Con_i^j. FT^{pre_i} denotes the finish time of predecessor tasks (t_{pre_i}) of t_i. TT^{pre_i} is the data transmission time from the predecessor task.

Let req_r represent a user request for a microservice application, the response time RT_r of req_r is calculated by:

$$RT_r = FT_{end} \tag{4}$$

As shown in Fig. 2, each container is deployed in a VM instance, while each VM is deployed in a PM instance [14], [15], [46], [50]. A VM/PM instance can host multiple container/VM instances. $VM_k = (vmcpu_k, price_k)$ represents a VM instance, where $vmcpu_k$ denotes the amount of vCPU provided by VM_k (i.e., CPU capacity) and $price_k$ indicates the hourly rental fee. The CPU capacity of a PM instance constrains the total CPU capacity of the VMs deployed in it, which further limits the available amount of vCPU provisioned to containers deployed in those VMs. The rental fee $Cost_k$ of any VM instance VM_k is calculated by:

$$Cost_k = price_k \times \frac{FT_{last}^k - ST_{first}^k}{3600}, \tag{5}$$

where FT_{last}^k and ST_{first}^k are the finish time and the start time of the last task and the first task executed in VM_k, respectively. The total cost $Cost(T)$ of renting VMs over a period of time T is calculate by:

$$Cost(T) = \sum_{k \in ACT_{VM}(T)} Cost_k, \tag{6}$$

where $ACT_{VM}(T)$ is the index set of active VMs over T.

C. Optimization Objective

In this article, we evaluate the QoS of microservice applications by *Average Response Time* (ART) [4], [10], [44]. Therefore, the aim of the AMC problem is to autoscaling containers to minimize the ART over a time period T while the $Cost(T)$ is under a cost budget, which is formulated as

$$\min ART(T) = \min \frac{\sum_{r \in REQ(T)} RT_r}{num} \tag{7}$$

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:11 UTC from IEEE Xplore. Restrictions apply.

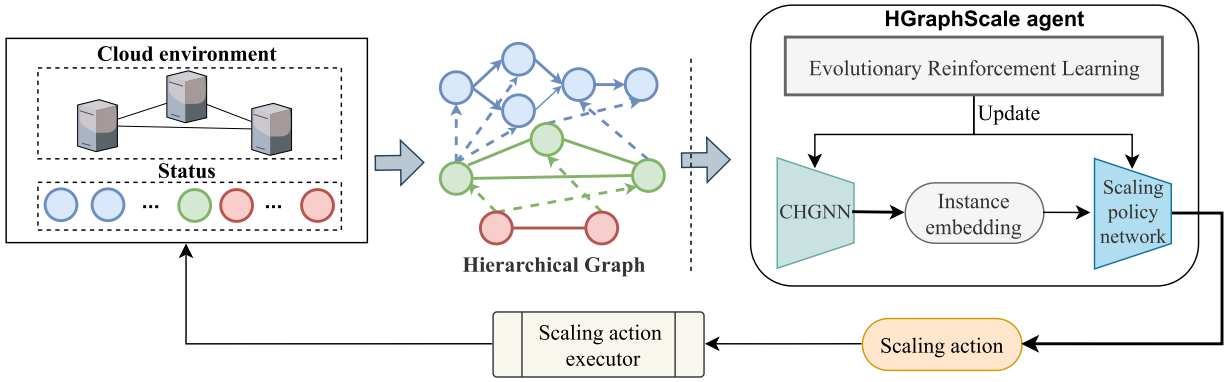

Fig. 3. The overall framework of HGraphScale for the AMC problem.

$$\text{s.t.} \quad Cost(T) \leq budget(T) \tag{8}$$

where $REQ(T)$ is the index set of user requests over T and num is the number of requests. $budget(T)$ is the cost budget of an application provider given over T.

The number of user requests sent to microservice applications varies over time. Thus, the autoscaling process needs to dynamically 1) identify the containers that require scaling, and 2) determine the optimal amount of scaling resources.

IV. Proposed Autoscaling Approach

The details of HGraphScale are introduced in this section. Specifically, we model the process of the AMC problem as a Reinforcement Learning (RL) problem in Section IV-A. The overall framework of HGraphScale is shown in Fig. 3. In each iteration, the state of the container-based cloud is extracted and represented as a hierarchical graph, detailed in Section IV-B. Then, a novel *Cloud-oriented Hierarchical Graph Neural Network (CHGNN)* is proposed to learn the embedding of every container, introduced in Section IV-C. The learned container embedding is fed into a newly designed scaling policy network to produce scaling actions, as described in Section IV-D. A *scaling action executor* performs either vertical or horizontal scaling in the container-based cloud based on the scaling actions, as outlined in Section IV-E. Section IV-F presents how HGraphScale handles load balancing.

Evolutionary Reinforcement Learning (ERL) [25], [40], a widely recognized and practically popular algorithm, is leveraged to train the neural networks of HGraphScale. This is because ERL demonstrates strong exploration ability, ensures a stable training process, and requires relatively few hyperparameters for fine-tuning [25], [40]. Moreover, recent studies have shown its effectiveness in several cloud-related applications [25], [41]. The detailed process of training by ERL is provided in Section IV-G.

A. RL Formulation

We formulate the process of solving AMC problems as an RL problem. Particularly, at each decision step t, the cloud environment provides the state s_t as a hierarchical graph. The HGraphScale agent in Fig. 3 generates a scaling action a_t based on s_t. The environment then performs a_t and transitions to the next state s_{t+1}. The key components of this RL problem are outlined below.

1) State: Each state s_t is a snapshot of the status of the PMs, VMs and containers in the container-based cloud at a decision step t. We design a novel hierarchical graph $\mathcal{H} = \langle \mathcal{V}, \mathcal{E} \rangle$ to represent the states s_t, detailed in Section IV-B. The number of nodes in this hierarchical graph dynamically changes with the addition or removal of containers.

The status of a PM instance p is defined as $h_p^{\vec{pm}} = \{\mu_{pm_p}, \Omega_{pm_p}\}$, which denotes the resource utilization (μ_{pm_p}) and the capacity (Ω_{pm_p}) of pm_p, respectively. Status of a VM instance v is defined as $h_v^{\vec{vm}} = \{\mu_{vm_v}, \Omega_{vm_v}, price_{vm_v}, rental_{vm_v}, art_{vm_v}\}$, which indicates the resource utilization (μ_{vm_v}), the capacity (Ω_{vm_v}), the per hour price ($price_{vm_v}$), the current rental fees ($rental_{vm_v}$) of vm_v, and the average response time (art_{vm_v}) of containers that are deployed in vm_v.

Similarly, the status of a container c is defined as $h_c^{\vec{con}} = \{\Omega_{con_c}, \zeta_{con_c}, d_{con_c}, pending_{con_c}, art_{con_c}, predicted_{con_c}\}$. Specifically, Ω_{con_c} denotes the resource capacity of con_c. ζ_{con_c} represents the remaining resources of the VM that hosts con_c, indicating that container autoscaling is constrained by the resources of its hosting VM. The degree of con_c in the graph is denoted as d_{con_c}. Moreover, $pending_{con_c}$ denotes pending requests, art_{con_c} the average response time, and $predicted_{con_c}$ the future workload of con_c.

This article follows [11] to employ an effective and efficient workload predicting method, i.e., the SMA method, to predict the number of future requests $predicted_{con_i}$ for a container con_i. The predicted future workload is based on the information from the historical workload.

2) Action: A scaling action a_t at time t of *HGrapScale* is represented as a 2-dimensional tuple: $\langle Ind, Scale \rangle$. $Ind \in [0, n] \cap \mathbb{Z}^+$ denotes the index of the container that requires scaling. Here, n is the current number of containers, which changes dynamically over time. $Scale \in [-m, +m] \cap \mathbb{Z}^+$ indicates the amount of resources for scaling. The sign of $Scale$ determines

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:11 UTC from IEEE Xplore. Restrictions apply.

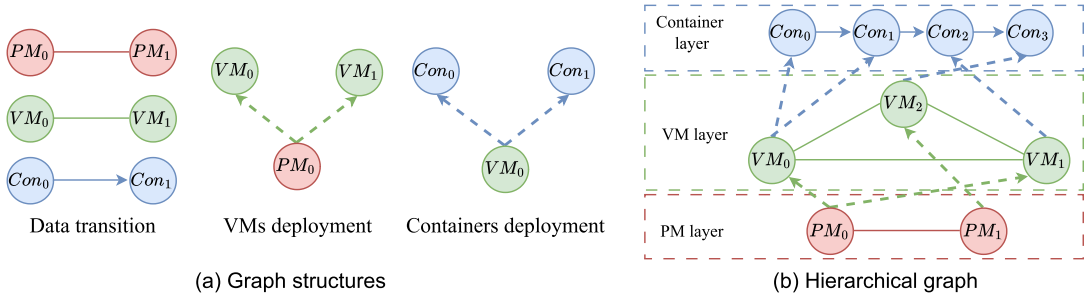

Fig. 4. An example of the hierarchical graph representation of the container-cloud.

whether to increase or decrease the provisioned resources for container Ind. If $Scale$ equals 0, it indicates that the resource provisioned to the Ind container remains unchanged.

3) Optimization Objective: To minimize the $ART(t)$ and ensure the cost adheres to the budget, the optimization objective of this RL problem is defined as

$$Obj(T) = -ART(T) - \rho \cdot \max\left(0, (Cost(T) - budget(T))\right) \tag{9}$$

where $Obj(T)$ is the objective value over a time period T and ρ controls the penalty intensity when the cost exceeds the given budget.

B. Hierarchical Graph Representation

To capture the hierarchical structure of containers, VMs and PMs in container-based clouds, as well as the communication relationships among containers, we represent the cloud state using a hierarchical graph $\mathcal{H} = \langle \mathcal{V}, \mathcal{E} \rangle$. Node set $\mathcal{V} = \mathbb{C} \cup \mathbb{V} \cup \mathbb{P}$ consists of sets of container Nodes $\mathbb{C}$, VM nodes $\mathbb{V}$ and PM nodes $\mathbb{P}$. Edge set $\mathcal{E} = \mathbb{E}_{depvm} \cup \mathbb{E}_{depcon} \cup \mathbb{E}_{pm} \cup \mathbb{E}_{vm} \cup \mathbb{E}_{con}$, where $\mathbb{E}_{depvm}$ and $\mathbb{E}_{depcon}$ represent the deployment scheme, $\mathbb{E}_{pm}$, $\mathbb{E}_{vm}$ and $\mathbb{E}_{con}$ represent the data transition between machines.

In particular, Fig. 4(a) presents three main structures of $\mathcal{H}$. The edge between two PM nodes $PM_0, PM_1 \in \mathbb{P}$ is undirected $\{PM_0, PM_1\} \in \mathbb{E}_{pm}$. It represents data transmission caused by interactions between containers running on different PMs, which causes additional communication overhead and thus affect QoS. Similarly, the edge between two VM nodes $VM_0, VM_1 \in \mathbb{V}$ is also undirected $\{VM_0, VM_1\} \in \mathbb{E}_{vm}$. For container nodes, there exist execution orders between connected containers. Thus, the edge between two container nodes $Con_0, Con_1 \in \mathbb{C}$ is directed, i.e., $(Con_0, Con_1) \in \mathbb{E}_{con}$. Directed edges $(PM_0, VM_0), (PM_0, VM_1) \in \mathbb{E}_{depvm}$ in the *VMs deployment* structures, indicating VM_0 and VM_1 are deployed in PM_0. Likewise, if Con_0 and Con_1 are deployed in VM_0, there are directed edges $(VM_0, Con_0), (VM_0, Con_1) \in \mathbb{E}_{depcon}$.

When a new container is joined by autoscaling, a new container node is added to the container layer. If the new container requires a new VM/PM for deployment, new VM/PM nodes are added to the VM/PM layer accordingly. Edges are then established among the container, VM, and PM nodes based on the deployment relationships.

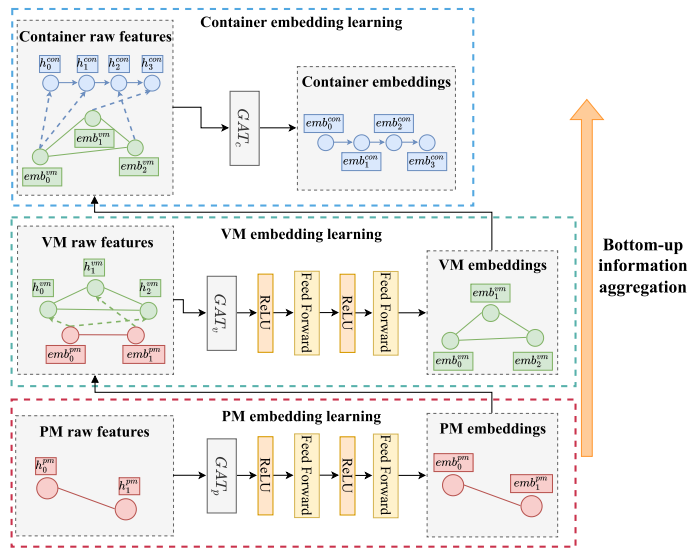

Fig. 5. The architecture of the CHGNN.

At each decision step t, the hierarchical graph (Fig. 4(b)) serves as the system state s_t for autoscaling decisions. The hierarchical graph consists of the *PM layer*, *VM layer* and *Container layer*. We denote PM features as $\mathbf{h^{pm}} = \{h_0^{\vec{pm}}, \ldots, h_P^{\vec{pm}}\}$, VM features as $\mathbf{h^{vm}} = \{h_0^{\vec{vm}}, \ldots, h_V^{\vec{vm}}\}$, and container features as $\mathbf{h^{con}} = \{h_0^{\vec{con}}, \ldots, h_C^{\vec{con}}\}$. The values P, V, and C correspond to the numbers of PMs, VMs, and containers, respectively. Our newly designed CHGNN learns container embedding from this hierarchical graph and the raw features of each node.

C. Cloud-Oriented Hierarchical Graph Neural Network

Given the hierarchical graph represented state, we proposed CHGNN to learns container embedding progressively through a bottom-up information aggregation mechanism, as shown in Fig. 5. Specifically, HCGNN first learns PM embedding in the *PM layer*, then PM embedding is propagated to the *VM layers* for VM embedding learning. At last, VM embedding is propagated to the *Container layer* for container embedding learning. Details of embedding learning in each layer and the bottom-up information aggregation are provided as follows.

1) Machine Embedding Learning: We stack *graph attention layers* [48] to construct Graph Attention Networks (GATs) (i.e.,

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:11 UTC from IEEE Xplore. Restrictions apply.

GAT_p, GAT_v, and GAT_c in Fig. 5) to learn embeddings of PMs, VMs and containers. The input to each graph attention layer consists of a graph and its node features. It then applies attention weights to aggregate neighbor information, resulting in updated node features.

Consider an example of learning PM embeddings by GAT_p. The GAT_p dynamically assigns attention weights $\alpha_{i,j}$ to PM p_i and its neighbor p_j in the PM layer, indicating the importance of p_j's features to p_i [48]. The $\alpha_{i,j}$ is calculated by

$$\alpha_{i,j} = \frac{\exp\left(\text{LeakyReLU}(\vec{a}^T[\mathbf{W_p}h_i^{\vec{pm}} \| \mathbf{W_p}h_j^{\vec{pm}}])\right)}{\sum_{k\in\mathcal{N}_i}\exp\left(\text{LeakyReLU}(\vec{a}^T[\mathbf{W_p}h_i^{\vec{pm}} \| \mathbf{W_p}h_k^{\vec{pm}}])\right)}, \tag{10}$$

where $\mathbf{W_p} \in \mathbb{R}^{F'\times F}$ is a learnable weight matrix in GAT_p, transforming the input features into high-level features. $\|$ indicates the concatenation operation. $\vec{a} \in \mathbb{R}^{2F'}$ is the learnable weight vector of a feedforward network. Following [48], LeakyReLU is applied for non-linear transformation. $\mathcal{N}_i$ is the neighbor nodes of p_i in the PM layer.

The updated features $h_i^{\vec{pm}'}$ of a p_i is generated by a graph attention layer according to

$$h_i^{\vec{pm}'} = \boldsymbol{\sigma}\left(\sum_{j\in\mathcal{N}_i}\alpha_{i,j}\mathbf{W_p}h_i^{\vec{pm}}\right), \tag{11}$$

where $\boldsymbol{\sigma}$ is the sigmoid activation function that enables modeling of nonlinearity. The GAT_p outputs node embedding $\mathbf{emb^{pm}} = \{emb_0^{pm}, \ldots, emb_P^{pm}\}$ of the PM layer after passing through multiple stacked *graph attention layers*. The VM layer and container layer follow the same process of embedding learning by GAT_v and GAT_c, respectively.

2) Bottom-Up Information Aggregation: In our proposed bottom-up information aggregation mechanism, the learned PM embedding $\mathbf{emb^{pm}} = \{emb_0^{pm}, \ldots, emb_P^{pm}\}$ propagates to the *VM layer*. Thus, the inputs of GAT_v are the concatenation of VM raw features and PM embedding $\mathbf{h^{vm}}\|\mathbf{emb^{pm}}$. The VM embedding $\mathbf{emb^{vm}} = \{emb_0^{vm}, \ldots, emb_V^{vm}\}$ are learned by the GAT_v and feed forward networks.

Similarly, the inputs of the GAT_c are the concatenation of container raw features and VM embedding $\mathbf{h^{con}}\|\mathbf{emb^{vm}}$, which outputs the container embedding $\mathbf{emb^{con}} = \{emb_0^{con}, \ldots, emb_C^{con}\}$. Through bottom-up information aggregation, the *Container layer* effectively incorporates global information of the container-based cloud into container embeddings. These embeddings allow the proposed *scaling policy network* to make system-aware scaling decisions.

D. Scaling Policy Network

To generate a scaling action, we design a *scaling policy network*, which takes the container embedding $\mathbf{emb^{con}} = \{emb_1^{con}, emb_2^{con}, \ldots, emb_C^{con}\}$ as input and outputs scaling actions, as illustrated in Fig. 6. A scaling action is defined as a tuple $\langle Ind, Scale\rangle$. To generate such actions, we design the scaling policy network with two MLPs: the *instance selector* MLP_ϕ and the *scale selector* MLP_ω.

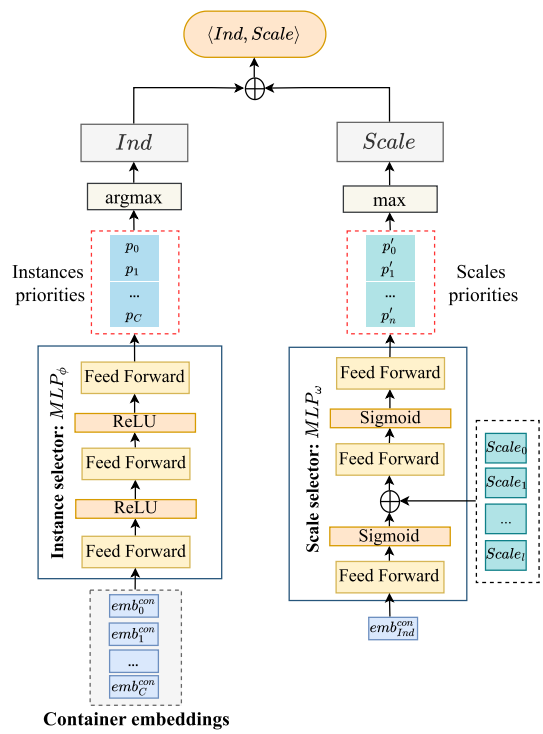

Fig. 6. The architecture of policy network.

MLP_ϕ is designed to calculate the priority values of each container i:

$$p_i = MLP_\phi\left(emb_i^{con}\right). \tag{12}$$

A container with a higher priority value implies a greater need for scaling. Thus, the index of the container to be scaled (e.g., Ind) is identified by

$$Ind = \underset{i=1,2,\ldots,C}{\arg\max}\left(p_i\right), \tag{13}$$

where C is the current number of containers.

After identifying the container for scaling, the corresponding container embedding $emb_{Ind}^{con} \in \mathcal{R}^{1\times d}$ is selected and fed into MLP_ω. As shown in Fig. 6, emb_{Ind}^{con} is passed through a feedforward network. The output is then concatenated with a vector $\mathcal{S} = \{S_0, S_1, \ldots, S_l\}$, where $S_j \in \mathbb{Z}$ indicates the amount of scaling resource, resulting in a new vector $\mathcal{I} = \{emb_{Ind}^{con}\|S_0, emb_{Ind}^{con}\|S_1, \ldots, emb_{Ind}^{con}\|S_l\}$. $\mathcal{I}$ is further processed by feed-forward networks. Finally, MLP_ω outputs the priority p_j' of each $Scale_j \in \mathcal{S}$. Therefore, the amount of scaling resources is determined by

$$Scale = \max_{j=0,1,2,\ldots,l}(p_j). \tag{14}$$

Afterwards, Ind and $Scale$ are combined to create a complete scaling action $\langle Ind, Scale\rangle$.

E. Scaling Action Executor

The scaling action executor transforms the scaling action $\langle Ind, Scale\rangle$ to vertical scaling, horizontal scaling or both.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:11 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Scaling Action Executor.

Input: Scaling action: $\langle Ind, Scale \rangle$
Output: vertical scaling or horizontal scaling
1: $tar_con \leftarrow container_list[Ind]$
2: $tar_vm \leftarrow$ the VM that hosts tar_con
3: **if** $Scale > 0$ **then**
4: $max_vcpu \leftarrow$ the remaining vCPUs of tar_vm
5: **if** $max_vcpu > Scale$ **then** ▷ vertical scaling
6: Increase $Scale$ vCPUs to tar_con
7: **else** ▷ horizontal scaling
8: Increase max_vcpu vCPUs to tar_con
9: $vcpu \leftarrow Scale - max_vcpu$
10: Create a new container with $vcpu$ vCPUs
11: **end if**
12: **else**
13: $con_vcpu \leftarrow$ number of vCPUs provisioned to tar_vm
14: **if** $con_vcpu > Scale$ **then** ▷ vertical scaling
15: Decrease $Scale$ vCPUs to tar_con
16: **else** ▷ thorizontal scaling
17: Delete tar_con
18: **end if**
19: **end if**

Algorithm 1 summarizes the process of scaling action executor. Firstly, a container tar_con is selected based on Ind (line 1).

If $Scale > 0$, the scaling action executor increases the resource provisioned to tar_con (lines 3 to 12). To be specific, if the remaining CPU capacity max_vcpu of the VM hosting the tar_con is larger than the $Scale$, vertical scaling is applied to provision $Scale$ vCPUs to the container (line 6); otherwise, max_vcpu vCPUs are provisioned to tar_con, then a new container is created by horizontal scaling. The newly created container is provisioned $(Scale - max_vcpu)$ vCPUs (lines 7 to 11).

The scaling action executor reduces the resource provisioned to tar_con when $Scale < 0$ (lines 12 to 19). If $Scale$ is larger than the total vCPUs of tar_con, the vCPUs of tar_con are reduced by the $Scale$ number (line 15). Otherwise, the container tar_con is deleted as a result of horizontal scaling (line 17). Note that the scaling action executor allows a microservice to be encapsulated within containers with heterogeneous resources, which can reduce resource wastage [45], [54]. As a result, a load balancer is implemented to dispatch user requests among heterogeneous containers, as detailed in Section IV-F.

F. Capacity-Based Load Balancing

HGraphScale applies Capacity-based Weighted Round-Robin (CWRR) [6], [24], [39], [44] to dispatch user requests to a suitable container for the purpose of load balancing. Specifically, the weight $\mathcal{W}_j$ of a container Con_i^j is determined by

$$\mathcal{W}_i^j = \frac{\gamma_i^j}{\sum_{k \in set(ms_i)} \gamma_i^k}, \tag{15}$$

Algorithm 2: Evolutionary Reinforcement Learning (ERL).

Input: Population size: N, maximum generation: max_gen, initial policy parameters: $\hat{\theta}$, learning rate: η, multi-variance gaussian noise standard deviation: σ
Output: Trained neural network
1: $gen \leftarrow 0$
2: **while** $gen \leq max_gen$ **do**
3: **for** $i = 0$ to N **do**
4: Sample perturbation $\epsilon_i \sim \mathcal{N}(0, 1)$
5: Update the neural network by using $\theta_i \leftarrow \hat{\theta} + \sigma\epsilon_i$
6: Calculate Fitness $F(\theta_i)$ based on (9)
7: **end for**
8: Estimate policy gradient $\nabla_\theta \mathbb{E}_{\epsilon_i \sim \mathcal{N}(0,1)} F(\hat{\theta} + \sigma\epsilon_i)$
9: $\hat{\theta} \leftarrow \hat{\theta} + \sigma F(\hat{\theta} + \sigma\epsilon_i)$
10: **end while**

where γ_i^j indicates the resource allocation of Con_i^j and $set(ms_i)$ denotes the container set of microservice ms_i.

The rationale for adopting CWRR in HGraphScale is threefold. First, CWRR is widely employed in practice owing to its simplicity [39]. Second, CWRR demonstrates low computation overhead in handling load balancing. Third, it provides effective load balancing by dispatching more user requests to containers with higher capacities. Thus, CWRR can prevent any container from being heavily utilized, reducing long tail response times [6], [24].

G. Evolutionary Reinforcement Learning

In this article, we adapt ERL [40] to train the neural networks of HGraphScale. ERL is a population-based approach to estimate the gradients of neural networks. Algorithm 2 presents the pseudo-code of the ERL.

In particular, the CHGNN and scaling policy network of HGraphScale initial all trainable parameters $\hat{\theta} = \{\vec{a}, W_p, W_v, W_c, \phi, \omega\}$, randomly. Each iteration starts with sampling N perturbations $[\epsilon_i]_{i=0,1,\ldots,N}$ from standard gaussian distribution $\mathcal{N}(0, 1)$ (line 4). Then, a population of N individuals $[\theta_i]_{i=0,1,\ldots,N}$ is generated by adding noise to $\hat{\theta}$ (line 5).

The fitness of an individual θ_i is evaluated based on the optimization objectives defined in (9) (line 6), which is calculated by

$$F(\theta_i) = Obj(T). \tag{16}$$

Then, the parameters of the policy network are updated by the estimated gradient, which is the expectation of individuals' fitness (line 8). Specifically, the gradient is estimated by

$$\nabla_\theta \mathbb{E}_{\epsilon \sim \mathcal{N}(0,1)} F(\hat{\theta} + \sigma\epsilon) = \frac{1}{\sigma} \nabla_\theta \mathbb{E}_{\epsilon \sim \mathcal{N}(0,1)} \left[F(\hat{\theta} + \sigma\epsilon)\epsilon \right]$$
$$\approx \frac{1}{N\sigma} \sum_{i=1}^{N} \left[F(\theta + \sigma\epsilon_i)\epsilon_i \right]. \tag{17}$$

Finally, the policy parameters are updated by gradient descent (line 9).

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:11 UTC from IEEE Xplore. Restrictions apply.

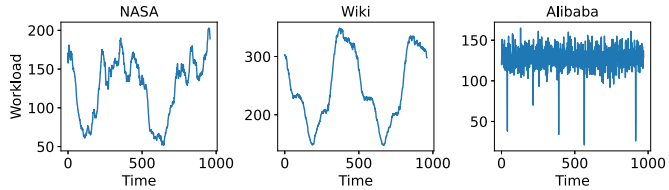

Fig. 7. Traces of user requests.

V. Performance Evaluation

In this section, we conduct comprehensive experiments to test the performance of our proposed HGraphScale. We first present the setup of experiments, the HGraphScale configuration and the competing approaches. Then, the experiment results are shown. Code of implementation, dataset and configuration are made publicly available.[1]

A. Experiment Setup

All experiments are carried out in a simulator that is implemented based on OpenAI Gymnasium [17]. The simulator models dynamic resource allocation across containers, VMs, and PMs, and reproduces fluctuating workload. It also simulates autoscaling behaviors with transient effects. The worst-case scenarios analysis in Appendix B enhances the fidelity of the simulator to real-world environments.

Three real-world traces of user requests, i.e., NASA,[2] Wiki[3] and Alibaba[4] are used to create workloads for our experiments. Fig. 7 illustrates the workload patterns over the 960-time-unit period (2 days) of NASA, Wiki and Alibaba, with each time unit representing a 3-minute interval. The workload trace patterns are shown in Fig. 7. The first 480 time units (one day) of workload from NASA or Wiki are extracted for training, while the remaining time units of workload are used for test [44]. In this article, a scaling action is made every 3 minutes, following [44].

Four medium-scale microservice applications [23], [42], [43] and a large-scale microservice application [11] are used for our experiments, as summarized in Fig. 8. Each microservice application has a different number of microservices and application structures. For convenience, we denote them as "A11", "A12", "A13", A14" and "A30", according to their microservices number.

Moreover, the cloud environment is equipped with 5 VM types from Amazon EC2.[5] The details of VM types are summarized in Table II. Each PM in the cloud environment has 64 vCPUs and 3200 GiB, following [46], [49].

To sum up, there are 15 scenarios designed for experiments based on three real-world traces and five types of microservice applications. In the initial stage of each scenario, each microservice is instantiated with a container, allocated with a vCPU and evenly deployed across three "m5.4xlarge" VMs [16], [46]. This setting allows each VM to have enough remaining resources to support further vertical scaling.

[1]https://github.com/sine-fandel/HGraphScale
[2]http://ita.ee.lbl.gov/html/traces.html
[3]http://www.wikibench.eu/wp-content/uploads/2010/10/vanbaaren-thesis.pdf
[4]https://github.com/alibaba/clusterdata/tree/master/cluster-trace-microservices-v2021
[5]https://aws.amazon.com/ec2/pricing/on-demand/

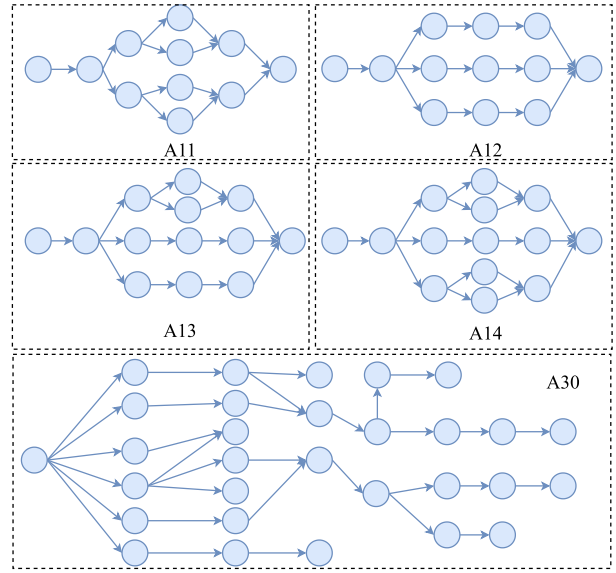

Fig. 8. Microservice applications used in experiments.

TABLE II
Five VM Types Used in Experiments

VM type	vCPU	Memory (GiB)	Hourly price ($)
m5.xlarge	4	16	0.192
m5.2xlarge	8	32	0.384
m5.4xlarge	16	64	0.768
m5.8xlarge	32	128	1.536
m5.12xlarge	48	192	2.304

B. HGraphScale Configuration

This article sets the number of *graph attention layers* as: in container layer $L_c = 2$, in VM layer $L_v = 1$ and PM layer $L_p = 1$, respectively. The dimension of GAT's output is 64. The hidden dimension of each feedforward network is set as 64.

All the hyperparameter settings of the ERL follow existing studies [25] that are designed for practical application. Specifically, we set the population size of the ERL as 40. The maximum generation is set as 1000, while HGraphScale converges at about 400 generations in all scenarios. The learning rate η and the Gaussian noise standard deviation of ERL σ are set as 0.01 and 0.05, respectively. The parameters are updated by Adam Optimizer. The $budget(T)$ of the optimization objective (9) is set as 200 USD per day [10], [57], while the performances under different budgets are evaluated in Section V-E. The penalty ρ is set as 100, following [44]. The performances of HGraphScale under different penalty settings are discussed in Section V-E4.

C. Competing Approaches

HGraphScale is compared to two heuristic-based autoscaling approaches, two state-of-the-art DRL-based autoscaling approaches and a GNN-based autoscaling approach. All competing

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:11 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PERFORMANCE COMPARISON IN TERMS OF ART AND VIO, WHERE VIO DENOTES THE AMOUNT BY WHICH THE COST EXCEEDS THE BUDGET (200 USD)

Scenario	AWS-Scale		ProScale		DeepScale		DRPC		AGQ		HGraphScale	
	ART	Vio	ART	Vio	ART	Vio	ART	Vio	ART	Vio	ART	Vio
NASA-11	410.4	0.00	305.5	104.1	306.6 ± 9.8	0.00	289.9 ± 7.3	0.00	278.1 ± 11.6	0.00	$\mathbf{255.1 \pm 6.4}$	0.00
NASA-12	688.5	0.00	387.7	45.7	532.6 ± 10.8	0.00	433.2 ± 8.1	20.8 ± 1.5	538.6 ± 15.1	26.7 ± 8.1	$\mathbf{268.4 \pm 11.2}$	0.00
NASA-13	899.0	0.00	406.8	1.6	493.6 ± 7.9	88.1 ± 11.0	532.3 ± 5.0	0.00	243.8 ± 6.7	0.00	$\mathbf{178.3 \pm 5.4}$	0.00
NASA-14	1022.1	0.00	532.3	56.7	348.0 ± 20.1	66.17%	510.7 ± 11.3	3.3 ± 2.0	336.4 ± 22.7	323.1 ± 60.1	$\mathbf{325.6 \pm 16.0}$	0.00
NASA-30	491.3	0.00	**303.4**	140.2	407.2 ± 7.1	0.00	391.9 ± 10.5	0.00	474.4 ± 8.8	3.39%	389.4 ± 17.2	0.00
Wiki-11	489.7	0.00	532.4	0.00	318.2 ± 11.2	69.4 ± 8.6	415.4 ± 7.7	56.3 ± 4.3	361.7 ± 21.1	82.0 ± 10.3	$\mathbf{307.7 \pm 15.4}$	0.00
Wiki-12	864.6	0.00	687.0	0.00	549.9 ± 19.0	0.00	512.4 ± 27.5	25.0 ± 2.0	457.0 ± 22.1	72.8 ± 16.6	$\mathbf{424.3 \pm 12.3}$	0.00
Wiki-13	1080.4	0.00	482.1	26.3	675.3 ± 21.1	0.00	491.6 ± 27.2	112.3 ± 24.3	$\mathbf{367.6 \pm 15.0}$	27.7 ± 5.5	369.1 ± 23.1	0.00
Wiki-14	1022.1	0.00	532.3	22.7	348.0 ± 19.1	53.2 ± 14.5	510.7 ± 20.1	0.00	520.2 ± 24.0	22.5 ± 4.4	$\mathbf{325.6 \pm 14.1}$	0.00
Wiki-30	395.6	0.00	426.6	102.1	388.3 ± 55.7	112.2 ± 7.8	374.8 ± 62.1	0.00	488.2 ± 34.2	20.5 ± 6.9	$\mathbf{350.9 \pm 33.0}$	0.00
Alibaba-11	395.8	0.00	476.6	0.00	307.4 ± 24.1	0.00	249.6 ± 31.2	0.00	295.9 ± 27.6	0.00	$\mathbf{222.1 \pm 26.6}$	0.00
Alibaba-12	665.5	0.00	654.0	0.00	312.2 ± 22.3	132.4 ± 20.1	291.8 ± 24.1	0.00	292.1 ± 33.6	24.1 ± 4.7	$\mathbf{283.7 \pm 17.8}$	0.00
Alibaba-13	525.7	0.00	281.6	0.00	212.4 ± 19.2	111.4 ± 22.8	251.7 ± 27.9	0.00	237.9 ± 21.2	0.00	$\mathbf{178.9 \pm 20.1}$	0.00
Alibaba-14	988.7	0.00	549.3	0.00	327.0 ± 41.8	40.5 ± 6.0	$\mathbf{277.0 \pm 30.3}$	113.6 ± 38.0	421.8 ± 31.2	25.4 ± 2.6	299.2 ± 36.1	0.00
Alibaba-30	474.7	0.00	237.7	315.7	210.1 ± 22.7	25.5 ± 16.2	191.3 ± 23.7	68.2 ± 24.1	442.8 ± 26.1	0.00	$\mathbf{183.9 \pm 17.7}$	0.00

approaches and HGraphScale share the same initial placement of containers. Moreover, they deploy newly created containers from horizontal scaling into suitable VMs using the Best-Fit heuristic [27]. With this heuristic, each new container is placed on the VM with the least remaining capacity that can still satisfy its demand. This strategy improves VM utilization and reduces the overall cost.

AWS-Scale [2] is a threshold-based autoscaling approach. Referring to [36], [44], we set the upper threshold as 0.8 and the lower threshold as 0.6 for the CPU utilization of each container.

ProScale [11] is a heuristic-based proactive autoscaling method that leverages the SMA to predict future request workloads of each container. The horizontal scaling is made according to the predicted future workload.

DeepScale [44] is an autoscaling approach based on DQN. Specifically, it uses a deep neural network to make high-level decisions, i.e., increase, decrease and maintain the amount of resources provisioned to containers. Then, heuristics based on queue theory are proposed to make low-level scaling actions, including horizontal scaling and vertical scaling.

DRPC [4] is a distributed reinforcement learning approach for autoscaling. It first trains a central module using TD3. After training the central module, multiple deployment units are trained to imitate the central module's behaviors. Deployment units make scaling actions (horizontal scaling and vertical scaling) for each microservice in a distributed manner.

AGQ [31] applies a Graph Convolution Network (GCN) for resource estimation. The predicted future resource demand is utilized to make horizontal scaling decisions, i.e., increase replicas, reduce replicas and no operation. The resource adjustment agent is trained by Q-learning.

We conduct 10 independent runs for each approach. Within each run, all approaches are evaluated sharing the same random seed, while different runs use different seeds.

D. Performance Comparison

Table III presents the test results on each scenario, where the best performance of ART in each scenario is highlighted in **bold**. Specifically, HGraphScale decreases from 37.17 When compared to DeepScale, HGraphScale produces 3.33 Table III also presents the violation degree ("Vio") that quantifies the percentage of cost exceeding the predefined budget (200 USD). We can observe from this table that the total VM rental cost of HGraphScale is always kept under the budget. This indicates that HGraphScale can make suitable scaling actions to avoid resource wastage. In contrast, ProScale, DeepScale, DRPC and AGQ exceed the budget in multiple scenarios. AWS-Scale also prevents budget violation by removing containers promptly when their CPU utilization is under the lower threshold. However, this design makes AWS-Scale vulnerable to QoS degradation under dynamic workloads. Container removal during low request periods leads to increased ART when user demand rises abruptly.

The observed performance improvements of HGraphScale can be attributed to our hierarchical representation and CHGNN design. These designs enable each container to effectively capture global information from VMs and PMs. This delivers a more precise autoscaling decision than prior studies [2], [4], [11], [31], [44], which improves QoS while simultaneously reducing resource waste and cost.

E. Further Analysis

1) Tail Response Time: Besides the ART, the tail response time also provides insights into the QoS of microservice applications in the industry [4], [13], [44], [56]. Fig. 9 shows the maximum response times at different percentiles of user requests in NASA-13 (other scenarios have similar trends). We can see that HGraphScale achieves lower response times at all percentiles. Fig. 10 provides the details of the response time distribution of HGraphScale in NASA-13. The results show that HGraphScale ensures 95.

2) Ablation Studies: To evaluate the effectiveness of the hierarchical graph learning, we conduct ablation studies by removing the PM layer of HGraphScale, giving rise to a variant named *w/o PM*. Moreover, we design another variant of HGraphScale without both VM and PM layers, named *w/o VM & PM*.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:11 UTC from IEEE Xplore. Restrictions apply.

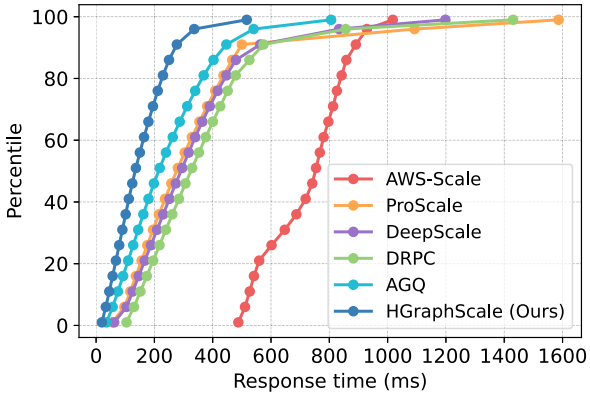

Fig. 9. Response times of NASA-13 at different percentiles for AWS-Scale, ProScale, DeepScale, DRPC and HGraphScale.

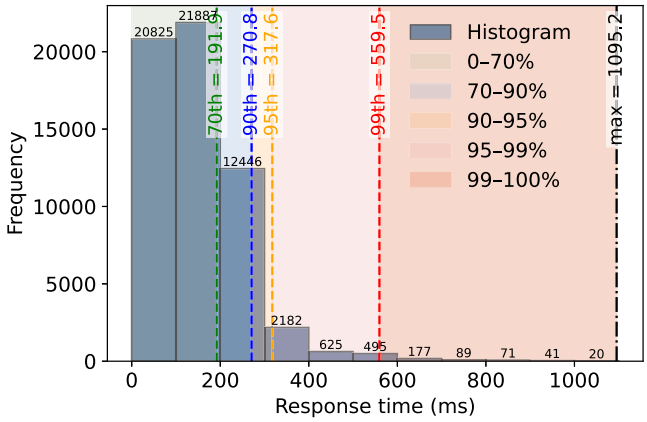

Fig. 10. Response time distribution of HGraphScale in NASA-13.

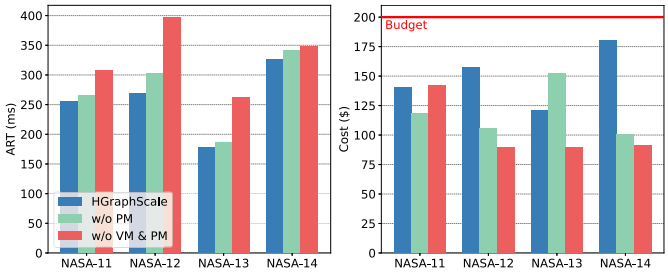

Fig. 11. The comparison results of ablation studies under NASA workload.

HGraphScale is compared with *w/o PM* and *w/o VM & PM* on NASA-11, NASA-12, NASA-13 and NASA-14.

As shown in Fig. 11, both *w/o PM* and *w/o VM & PM* ensure the cost is not exceed the budget. However, *w/o PM* exhibits significantly inferior ART compared to HGraphScale, with *w/o VM & PM* performing even worse than *w/o PM*. These results indicate the effectiveness of both VM and PM embedding learning in HGraphScale.

3) Performance Comparison With Different Budget: We compare the performance of HGraphScale in solving the AMC problem with different cost budgets, that is, $150 and 250$. Table IV presents the ART and cost under different budgets. Specifically, both the stringent and relaxed budgets of the AMC problem can be satisfied by HGraphScale. We observe that the HGraphScale achieves lower ART under $250 budgets than under 150$ budgets. The reason is that a relaxed budget allows for the provision of more resources to the containers.

TABLE IV
PERFORMANCE COMPARISON WITH DIFFERENT BUDGET: 150$ AND 250$

ρ	NASA-11 ART (ms)	NASA-11 Cost ($)	NASA-13 ART (ms)	NASA-13 Cost ($)
50	273.5 ± 17.2	145.5 ± 9.3	169.4 ± 18.7	133.7 ± 20.2
100	255.1 ± 6.4	140.6 ± 7.2	178.3 ± 5.4	120.6 ± 11.9
150	256.4 ± 16.1	117.3 ± 8.8	180.1 ± 21.2	146.2 ± 15.5
200	324.0 ± 17.0	148.2 ± 8.2	203.6 ± 12.9	157.0 ± 5.5

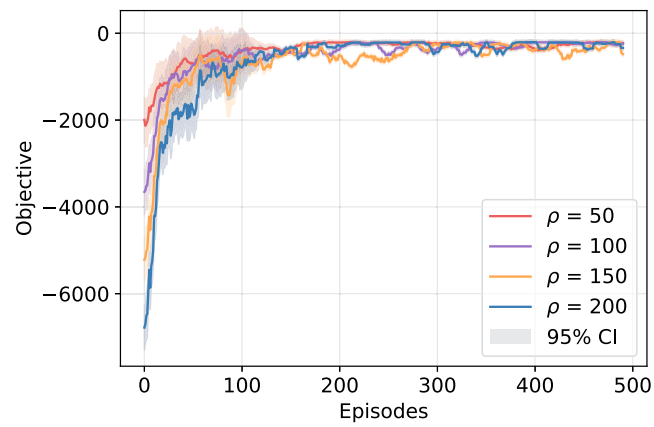

Fig. 12. Training curve of HGraphScale under different settings of penalty ρ on NASA-13.

TABLE V
PERFORMANCE COMPARISON WITH PENALTY COEFFICIENT (ρ)

ρ	NASA-11 ART (ms)	NASA-11 Cost ($)	NASA-13 ART (ms)	NASA-13 Cost ($)
50	273.5 ± 17.2	145.5 ± 9.3	169.4 ± 18.7	133.7 ± 20.2
100	255.1 ± 6.4	140.6 ± 7.2	178.3 ± 5.4	120.6 ± 11.9
150	256.4 ± 16.1	117.3 ± 8.8	180.1 ± 21.2	146.2 ± 15.5
200	324.0 ± 17.0	148.2 ± 8.2	203.6 ± 12.9	157.0 ± 5.5

4) Performance Comparison With Different Penalty: The optimization objective of HGraphScale includes a penalty term (ρ) for violating the budget. Therefore, we conduct sensitivity analysis on different penalty settings, that is $\rho = 50$, $\rho = 100$, $\rho = 150$ and $\rho = 200$. Fig. 12 illustrates the training curves on NASA-13 obtained by different penalty settings. We can observe from this figure that the training process of HGraphScale is robust to different penalty settings, as they all achieve similar convergence stability.

Table V presents the test performance of HGraphScale. HGraphScale ensures the cost under 200$/day with different settings of ρ. Moreover, when $\rho = 50$, $\rho = 100$, and $\rho = 150$, HGraphScale achieves similar performances in terms of ART,

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:11 UTC from IEEE Xplore. Restrictions apply.

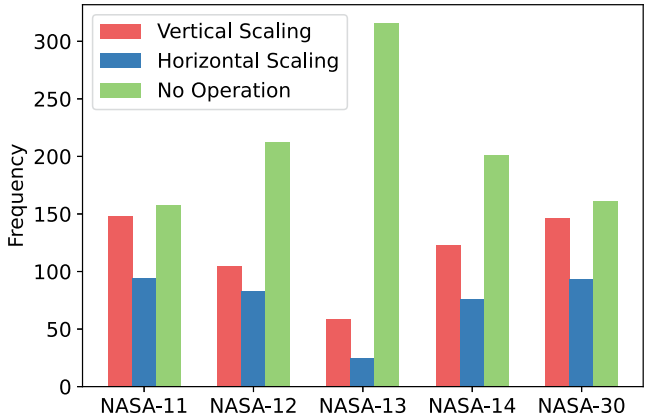

Fig. 13. Quantitative breakdown of HGraphScale's scaling actions.

while performance degradation occurs with $\rho = 200$. This is because the harsh penalty limits the exploration ability of HGraphScale during training.

5) Quantitative Analysis of Scaling Actions: To better understand the behavior of HGraphScale, we conduct a detailed analysis of a quantitative breakdown of autoscaling actions. Fig. 13 demonstrates the frequencies of scaling actions generated by HGraphScale, including *vertical scaling*, *horizontal scaling*, and *no operation*. This figure provides evidence that HGraphScale tends to perform more vertical scaling than horizontal scaling in each scenario, resulting in fewer container replicas.

Moreover, Fig. 13 also shows that *no operation* dominates in all scenarios. These results indicate that HGraphScale improves application performance while maintaining system stability without frequent scaling. It further demonstrates HGraphScale's ability to accurately identify containers requiring scaling and to determine appropriate scaling levels, thereby avoiding resource wastage.

VI. Conclusion and Future Work

In this article, we propose HGraphScale, a novel DRL-based autoscaling approach for microservice applications in a container-based cloud. Particularly, we propose a hierarchical graph to capture dependencies in container-based clouds, a CHGNN with bottom-up aggregation to learn container embeddings, and a scaling policy network that makes scaling decisions based on these embeddings. The experimental results indicate that HGraphScale reduces average response time compared to threshold-based, DRL-based, and graph-based autoscaling, without exceeding the cost budget. In future work, we will investigate multi-resource autoscaling to further enhance our method. Meanwhile, we will consider the container migration and respawning to further reduce cost by enabling more efficient VM utilization.

References

[1] Y. Al-Dhuraibi, F. Paraiso, N. Djarallah, and P. Merle, "Elasticity in cloud computing: State of the art and research challenges," *IEEE Trans. Serv. Comput.* vol. 11, no. 2, pp. 430–447, Mar./Apr. 2018.

[2] Amazon: AWS Auto Scaling, 2022. [Online]. Available: https://aws.amazon.com/autoscaling/

[3] A. F. Baarzi and G. Kesidis, "SHOWAR: Right-sizing and efficient scheduling of microservices," in *Proc. ACM Symp. Cloud Comput.*, 2021, pp. 427–441.

[4] H. Bai, M. Xu, K. Ye, R. Buyya, and C. Xu, "DRPC: Distributed reinforcement learning approach for scalable resource provisioning in container-based clusters," *IEEE Trans. Serv. Comput.*, vol. 17, no. 6, pp. 3473–3484, Nov./Dec. 2024.

[5] L. Bao, C. Wu, X. Bu, N. Ren, and M. Shen, "Performance modeling and workflow scheduling of microservice-based applications in clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 9, pp. 2114–2129, Sep. 2019.

[6] L. Baresi, D. Y. X. Hu, G. Quattrocchi, and L. Terracciano, "KOSMOS: Vertical and horizontal resource autoscaling for kubernetes," in *Proc. Int. Conf. Service-Oriented Comput.*, 2021, pp. 821–829.

[7] V. Berry, A. Castelltort, B. Lange, J. Teriihoania, C. Tibermacine, and C. Trubiani, "Is it worth migrating a monolith to microservices? An experience report on performance, availability and energy usage," in *Proc. IEEE Int. Conf. Web Serv.*, 2024, pp. 944–954.

[8] G. Blinowski, A. Ojdowska, and A. Przybyłek, "Monolithic vs. microservice architecture: A performance and scalability evaluation," *IEEE Access*, vol. 10, pp. 20357–20374, 2022.

[9] B. Burns, J. Beda, and K. Hightower, *Kubernetes: Up and Running: Dive Into the Future of Infrastructure*. Sebastopol, CA, USA: O'Reilly Media, 2019.

[10] K. Cheng et al., "GeoScale: Microservice autoscaling with cost budget in geo-distributed edge clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 4, pp. 646–662, Apr. 2024.

[11] K. Cheng et al., "ProScale: Proactive autoscaling for microservice with time-varying workload at the edge," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 4, pp. 1294–1312, Apr. 2023.

[12] S. Chouliaras and S. Sotiriadis, "An adaptive auto-scaling framework for cloud resource provisioning," *Future Gener. Comput. Syst.*, vol. 148, pp. 173–183, 2023.

[13] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[14] Z. Fang, H. Ma, G. Chen, and S. Hartmann, "Energy-efficient and communication-aware resource allocation in container-based cloud with group genetic algorithm," in *Proc. Int. Conf. Service-Oriented Comput.*, 2023, pp. 212–226.

[15] Z. Fang, H. Ma, G. Chen, and S. Hartmann, "A group genetic algorithm for energy-efficient resource allocation in container-based clouds with heterogeneous physical machines," in *Proc. Australas. Joint Conf. Artif. Intell.*, 2023, pp. 453–465.

[16] Z. Fang, H. Ma, G. Chen, S. Hartmann, and S. Chen, "A communication-aware and energy-efficient genetic programming based method for dynamic resource allocation in clouds," in *Proc. Int. Conf. Appl. Evol. Comput. (Part EvoStar)*, 2025, pp. 421–436.

[17] F. Foundation, "Gymnasium documentation, " 2025. [Online]. Available: https://gymnasium.farama.org/index.html

[18] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1587–1596.

[19] Y. Garí, D. A. Monge, and C. Mateos, "A Q-learning approach for the autoscaling of scientific workflows in the cloud," *Future Gener. Comput. Syst.*, vol. 127, pp. 168–180, 2022.

[20] Y. Garí, E. Pacini, L. Robino, C. Mateos, and D. A. Monge, "Online RL-based cloud autoscaling for scientific workflows: Evaluation of Q-learning and sarsa," *Future Gener. Comput. Syst.*, vol. 157, pp. 573–586, 2024.

[21] I. Hamzaoui, B. Duthil, V. Courboulay, and H. Medromi, "A topical review on container-based cloud revolution: Multi-directional challenges, and future trends," *SN Comput. Sci.*, vol. 5, no. 4, 2024, Art. no. 416.

[22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[23] K. C. Huang and B. J. Shen, "Service deployment strategies for efficient execution of composite SAAS applications on cloud platform," *J. Syst. Softw.*, vol. 107, pp. 127–141, 2015.

[24] V. Huang et al., "A scalable approach to SDN control plane management: High utilization comes with low latency," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 2, pp. 682–695, Jun. 2020.

[25] V. Huang, C. Wang, H. Ma, G. Chen, and K. Christopher, "Cost-aware dynamic multi-workflow scheduling in cloud data center using evolutionary reinforcement learning," in *Proc. Int. Conf. Service-Oriented Comput.*, 2022, pp. 449–464.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:11 UTC from IEEE Xplore. Restrictions apply.

[26] M. Imdoukh, I. Ahmad, and M. G. Alfailakawi, "Machine learning-based auto-scaling for containerized applications," *Neural Comput. Appl.*, vol. 32, no. 13, pp. 9745–9760, 2020.

[27] S. Jangiti, V. Vijayakumar, and V. Subramaniyaswamy, "Hybrid best-fit heuristic for energy efficient virtual machine placement in cloud data centers," *EAI Endorsed Trans. Energy Web*, vol. 7, 2020, Art. no. 26.

[28] B. Jeong and Y. S. Jeong, "Autoscaling techniques in cloud-native computing: A comprehensive survey," *Comput. Sci. Rev.*, vol. 58, 2025, Art. no. 100791.

[29] S. Lee and J. Park, "Comparative performance analysis of I/O interfaces on different NVMe SSDs in a high cpu contention scenario," *IEICE Trans. Inf. Syst.*, vol. 107, no. 7, pp. 898–900, 2024.

[30] T. Li, S. Ying, X. Tian, T. Zhang, and Y. Wang, "ASTRA: Adversarial sim-to-real transfer reinforcement learning for autoscaling in cloud systems," *IEEE Trans. Softw. Eng.*, vol. 51, no. 10, pp. 2921–2941, Oct. 2025.

[31] P. Liang, Y. Xun, J. Cai, and H. Yang, "Autoscaling of microservice resources based on dense connectivity spatio-temporal GNN and Q-learning," *Future Gener. Comput. Syst.*, 174, 2026, Art. no. 107909.

[32] Y. Ma, P. Gerard, Y. Tian, Z. Guo, and N. V. Chawla, "Hierarchical spatio-temporal graph neural networks for pandemic forecasting," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, 2022, pp. 1481–1490.

[33] C. Meng, S. Song, H. Tong, M. Pan, and Y. Yu, "DeepScaler: Holistic autoscaling for microservices based on spatiotemporal gnn with adaptive graph learning," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2023, pp. 53–65.

[34] C. Meng, J. Tong, M. Pan, and Y. Yu, "HRA: An intelligent holistic resource autoscaling framework for multi-service applications," in *Proc. IEEE Int. Conf. Web Serv.*, 2022, pp. 129–139.

[35] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature* vol. 518, no. 7540, pp. 529–533, 2015.

[36] S. M. R. Nouri, H. Li, S. Venugopal, W. Guo, M. He, and W. Tian, "Autonomic decentralized elasticity based on a reinforcement learning controller for cloud applications," *Future Gener. Comput. Syst.*, vol. 94, pp. 765–780, 2019.

[37] J. Park, B. Choi, C. Lee, and D. Han, "GRAF: A graph neural network based proactive resource allocation framework for slo-oriented microservices," in *Proc. 17th Int. Conf. Emerg. Netw. Exp. Technol.*, 2021, pp. 154–167.

[38] J. Park, B. Choi, C. Lee, and D. Han, "Graph neural network-based SLO-aware proactive resource autoscaling framework for microservices," *IEEE/ACM Trans. Netw.*, vol. 32, no. 4, pp. 3331–3346, Aug. 2024.

[39] I. Saidu, S. Subramaniam, A. Jaafar, and Z. A. Zukarnain, "A load-aware weighted round-robin algorithm for ieee 802.16 networks," *EURASIP J. Wireless Commun. Netw.*, vol. 2014, no. 1, 2014, Art. no. 226.

[40] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, "Evolution strategies as a scalable alternative to reinforcement learning," 2017, *arXiv:1703.03864*.

[41] Y. Shen, G. Chen, H. Ma, and M. Zhang, "Cost-aware dynamic cloud workflow scheduling using self-attention and evolutionary reinforcement learning," in *Proc. Int. Conf. Service-Oriented Comput.*, 2024, pp. 3–18.

[42] T. Shi, H. Ma, G. Chen, and S. Hartmann, "Location-aware and budget-constrained service deployment for composite applications in multi-cloud environment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 8, pp. 1954–1969, Aug. 2020.

[43] T. Shi, H. Ma, G. Chen, and S. Hartmann, "Cost-effective web application replication and deployment in multi-cloud environment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 8, pp. 1982–1995, Aug. 2022.

[44] T. Shi, H. Ma, G. Chen, and S. Hartmann, "Auto-scaling containerized applications in GEO-distributed clouds," *IEEE Trans. Serv. Comput.*, vol. 16, no. 6, pp. 4261–4274, Nov./Dec. 2023.

[45] S. N. Srirama, M. Adhikari, and S. Paul, "Application deployment using containers with auto-scaling for microservices in cloud environment," *J. Netw. Comput. Appl.*, vol. 160, 2020, Art. no. 102629.

[46] B. Tan, H. Ma, Y. Mei, and M. Zhang, "A cooperative coevolution genetic programming hyper-heuristics approach for on-line resource allocation in container-based clouds," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1500–1514, Jul.–Sep. 2022.

[47] G. Tong, C. Meng, S. Song, M. Pan, and Y. Yu, "GMA: Graph multi-agent microservice autoscaling algorithm in edge-cloud environment," in *Proc. IEEE Int. Conf. Web Serv.*, 2023, pp. 393–404.

[48] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Li′o, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=rJXMpikCZ

[49] C. Wang, H. Ma, G. Chen, V. Huang, Y. Yu, and K. Christopher, "Energy-aware dynamic resource allocation in container-based clouds via cooperative coevolution genetic programming," in *Proc. Int. Conf. Appl. Evol. Comput. (Part EvoStar)*, 2023, pp. 539–555.

[50] S. Wang, Z. Ding, and C. Jiang, "Elastic scheduling for microservice applications in clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 1, pp. 98–115, Jan. 2021.

[51] S. Wang, X. Li, Q. Z. Sheng, and A. Beheshti, "Performance analysis and optimization on scheduling stochastic cloud service requests: A survey," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 3, pp. 3587–3602, Sep. 2022.

[52] C. J. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, pp. 279–292, 1992.

[53] L. Wen et al., "StatuScale: Status-aware and elastic scaling strategy for microservice applications," *ACM Trans. Auton. Adaptive Syst.*, vol. 20, no. 1, pp. 1–25, 2025.

[54] Z. Wen, Q. Chen, Q. Deng, Y. Niu, Z. Song, and F. Liu, "ComboFunc: Joint resource combination and container placement for serverless function scaling with heterogeneous container," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 11, pp. 1989–2005, Nov. 2024.

[55] N. Wu, X. W. Zhao, J. Wang, and D. Pan, "Learning effective road network representation with hierarchical graph neural networks," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 6–14.

[56] J. Xie, D. Guo, X. Li, Y. Shen, and X. Jiang, "Cutting long-tail latency of routing response in software defined networks," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 3, pp. 384–396, Mar. 2018.

[57] S. Xie, J. Wang, B. Li, Z. Zhang, D. Li, and P. C. Hung, "PBScaler: A bottleneck-aware autoscaling framework for microservice-based applications," *IEEE Trans. Serv. Comput.*, vol. 17, no. 2, pp. 604–616, Mar./Apr. 2024.

[58] Y. Yang, G. Chen, H. Ma, and M. Zhang, "Dual-tree genetic programming for deadline-constrained dynamic workflow scheduling in cloud," in *Proc. Int. Conf. Service-Oriented Comput.*, 2022, pp. 433–448.

[59] Z. Ying, J. You, C. Morris, X. Ren, W. Hamilton, and J. Leskovec, "Hierarchical graph representation learning with differentiable pooling," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 4805–4815.

[60] S. Zhang, T. Wu, M. Pan, C. Zhang, and Y. Yu, "A-SARSA: A predictive container auto-scaling algorithm based on reinforcement learning," in *Proc. IEEE Int. Conf. Web Serv.*, 2020, pp. 489–497.

[61] Z. Zhong, C. T. Li, and J. Pang, "Hierarchical message-passing graph neural networks," *Data Mining Knowl. Discov.*, vol. 37, no. 1, pp. 381–408, 2023.

[62] R. Zolfaghari, "Energy-performance aware virtual machines migration in cloud network by using prediction and fuzzy approaches," *Eng. Appl. Artif. Intell.*, vol. 131, 2024, Art. no. 107825.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:11 UTC from IEEE Xplore. Restrictions apply.